

AI Agents and Agentic AI with Python & Generative AI		✕
Module 1		↕
Agentic AI Concepts		
Module 2		↕
AI Agents, Tools, Actions, & Language		
Module 3		↕
GAME: A Conceptual Framework for AI Agents		
Module 4		↗
Agent Tool Management		
🟢	Keeping Agent Tools Up to Date with Python Decorators	15 min
🟢	Tool Organization for Agents	15 min
🟢	Refactoring OUR README Agent	15 min
🟢	Try Out the README Agent with the Decorator Reading	10 min
Module 5		↗
Rethinking How Software is Built in the Age of AI Agents		
🟢	Build the Impossible with AI Agents	Video - 1 min
🟢	Rethinking How We Teach Innovation	Video - 4 min
🟢	The Inventory Management Agent	Ungraded Playgen - 15 min
🟢	Hallucination is a New Form of Computing	Video - 7 min
🟢	New Ways to Access and Extract Information	Video - 3 min

Refactoring Our README Agent

Refactoring Our README Agent: Using Tool Decorators

In our original README agent, we manually created and registered each action. While this approach works, it requires us to maintain the tool definitions (functions) separately from their metadata (descriptions and parameters). This separation creates opportunities for these two pieces to become out of sync. Let's improve our agent by using tool decorators to keep everything together and automatically synchronized.

Understanding the Problem

Let's look at a piece of our original code:

```
def read_project_file(name: str) -> str:
    with open(name, "r") as f:
        return f.read()

# Later, separately, we define metadata about the function
action_registry.register(action={
    name="read_project_file",
    function=read_project_file,
    description="Reads a file from the project.",
    parameters={
        "type": "object",
        "properties": {
            "name": {"type": "string"}
        },
        "required": ["name"]
    },
    terminal=False
})
```

In this code, if we change the function's parameters, we need to remember to update the parameter schema. If we modify what the function does, we need to remember to update the description. These manual steps are error-prone and can lead to confusion for both developers and the LLM.

The Decorator Solution

Using tool decorators, we can keep all this information together. Here's how we'll refactor our README agent:

```

First, we'll define our tools using decorators
@register_tool(tags=["file_operations", "read"])
def read_project_file(name: str) -> str:
    """Reads and returns the content of a specified project file.

    Opens the file in read mode and returns its entire contents as a string.
    Raises FileNotFoundError if the file doesn't exist.

    Args:
        name: The name of the file to read

    Returns:
        The contents of the file as a string
    """
    with open(name, "r") as f:
        return f.read()

@register_tool(tags=["file_operations", "list"])
def list_project_files() -> list[str]:
    """Lists all Python files in the current project directory.

    Scans the current directory and returns a sorted list of all files
    that end with ".py".

    Returns:
        A sorted list of Python filenames
    """
    return sorted([file for file in os.listdir(".")
                    if file.endswith(".py")])

@register_tool(tags=["system", "terminal"]=True)
def terminate(message: str) -> str:
    """Terminates the agent's execution with a final message.

    Args:
        message: The final message to return before terminating

    Returns:
        The message with a termination note appended
    """
    return f"({message})Terminating..."

def main():
    # Define the agent's goals
    goals = [
        Goal(priority=1,
              name="Gather Information",
              description="Read each file in the project in order to build a deep
              understanding of the project in order to write a README"),
        Goal(priority=1,
              name="Terminate",
              description="Call terminate when done and provide a complete README
              for the project (message parameter)")
    ]

    # Create an agent instance with tag-filtered actions
    agent = Agent(
        goals=goals,
        agent_language=AgentFunctionCallActionLanguage(),
        # The ActionRegistry now automatically loads tools with these tags
        action_registry=PythonActionRegistry(tags=["file_operations",
                                                  "system"],
                                           generate_response=generate_response,
                                           environment=Environment())
    )

    # Run the agent with user input
    user_input = "Write a README for this project."
    final_memory = agent.run(user_input)
    print(final_memory.get_memories())

if __name__ == "__main__":
    main()

```

Let's examine the key improvements this refactoring brings:

1. Self-Documenting Tools

Each tool now carries its own documentation through Python docstrings. The decorator automatically extracts this documentation and uses it as the tool's description. This means:

- Documentation lives with the code it describes
- Changes to the function naturally lead to updates in documentation
- We get better IDE integration with docstring hints

2. Automatic Parameter Inference

The decorator examines the function's type hints and signature to automatically build the parameter schema. This means:

- Parameter types stay in sync automatically
- Required parameters are detected from the function signature
- We don't need to manually maintain a separate schema

3. Logical Organization Through Tags

Tools are now organized with tags that describe their purpose:

- `file_operations` for file-related tools
- `read` and `list` to specify the type of operation
- `system` for administrative functions like termination

This tagging system makes it easy to:

- Group related tools together
- Filter tools when creating agents
- Understand a tool's purpose at a glance

4. Simplified Agent Creation

Notice how much simpler our agent creation becomes:

```
agent = Agent(  
    goals=goals,  
    agent_language=AgentFunctionCallingActionLanguage(),  
    action_registry=ActionRegistry(tags=["file_operations", "system"]),  
    generate_response=generate_response,  
    environment=Environment()  
)
```

We no longer need to manually register actions. Instead, we just specify which tags we want, and the `ActionRegistry` automatically includes the appropriate tools.

5. Easier Maintenance

When we need to modify our agent's capabilities, we can:

- Add new tools by simply decorating functions
- Modify existing tools without touching registration code
- Change tool organization by adjusting tags
- Remove tools by removing the decorator or changing tags

Understanding the Flow

With this refactored version:

1. When the code loads, the decorators automatically register all tools in a central registry.
2. When we create an `ActionRegistry` with specific tags, it automatically loads the matching tools.
3. The Agent uses these pre-configured tools without any manual registration steps.

This automation reduces errors and makes our code more maintainable. If we later want to add new capabilities to our README agent, we simply need to create new functions with the appropriate decorators and tags.